

Lucrarea nr. 11: **Studiul și implementarea automatelor FSM** (FSM - Finite State Machine)

1. Scopul lucrării

Prezentarea modalităților de analiză și sinteză logică a automatelor de tip FSM și implementarea acestora în structuri de tip CPLD sau FPGA. Totodată se continuă seria de prezentarea prin exemple a limbajului de descriere hardware de tip VHDL.

2. Considerente teoretice

Automatele elementare sunt circuite logice secvențiale alcătuite dintr-un registru de memorie și un circuit logic combinațional.

Registrul de memorie, denumit și registru de stare, este format dintr-un număr de bistabili, de regulă de același tip, ale căror intrări de ceas sunt conectate împreună formând astfel intrarea de ceas a automatului (intrarea CK). *Starea unui automat este dată de valorile logice ale bistabililor din registrul de stare și, în consecință, este susceptibilă de modificare după fiecare tranziție activă a semnalului aplicat intrărilor de ceas.*

Circuitul logic combinațional are dublu rol: calculează ieșirile automatului și starea următoare a acestuia. *Starea următoare* a unui automat este calculată în funcție de starea prezentă (indicată de registrul de stare) și în funcție de starea semnalelor de intrare. Starea următoare înlocuiește stare prezentă imediat după ce apare prima tranziție activă a semnalului de ceas. În calculul semnalelor de ieșire, pe lângă starea prezentă a automatului, în funcție de tipul automatului, mai poate interveni starea semnalelor de intrare.

2. 1. Clasificarea automatelor sincrone

a) După modul de calcul al ieșirii:

- **Automate de tip Medvedev** – ieșirea automatului este identică cu starea automatului:

$$\text{ieșire} = F(\text{stare prezentă})$$

$$\text{stare viitoare} = G(\text{intrare, stare prezentă})$$

- **Automate de tip Moore** - ieșirea este dependentă numai de starea prezentă a automatului:

$$\text{ieșire} = F(\text{stare prezentă})$$

$$\text{stare viitoare} = G(\text{intrare, stare prezentă})$$

- **Automate de tip Mealy** - ieșirea este dependentă de intrare și de starea prezentă a automatului:

$$\text{ieșire} = F(\text{intrare, stare prezentă})$$

$$\text{stare viitoare} = G(\text{intrare, stare prezentă})$$

b) După modul de utilizare a ieșirii calculate:

- Automate cu ieșire imediată;
- Automate cu ieșire întârziată;

Schemele bloc de principiu ale automatelor de tip Mealy și Moore sunt prezentate în fig. 1.

Observații:

- Automatele cu utilizarea imediată a ieșirilor prezintă dezavantajul că fenomenele tranzitorii (de scurtă durată) ce apar imediat după tranziția activă a semnalului de ceas, sunt transmise direct la ieșirile automatului.
- Automatele cu întârziere prezintă în plus față de cele imediate un registru de memorie în care se stochează ieșirile calculate.
- Pentru automatele Mealy imediate, modificarea intrărilor are efect imediat asupra ieșirilor ceea ce înseamnă că modificarea ieșirilor are loc în același tact cu modificarea intrărilor.
- Pentru automatele Moore imediate, modificarea ieșirilor se face cu o întârziere de un tact în raport cu modificarea intrărilor. Intrările ce se modifică în tactul t , afectează starea curentă a automatului din tactul $t+1$, care mai departe determină schimbarea ieșirilor tot în tactul $t+1$. Deci, la ieșire, efectul intrărilor este resimțit cu o întârziere de o perioadă a semnalului de ceas, cu toate că automatul este referit ca un automat imediat.
- La automatele cu întârziere, ieșirile calculate sunt înscrise într-un registru de memorie suplimentar, altul decât cel de stare. Înscriserea se face la tactul următor al ceasului atunci când se presupune că regimul tranzitoriu este stins. În acest mod, față de cazul automatelor imediate, la cele cu întârziere, se mai adaugă o întârziere suplimentară de un tact. Astfel că răspunsul unui automat Mealy cu întârziere apare după un tact, iar pentru un automat Moore cu întârziere după două.

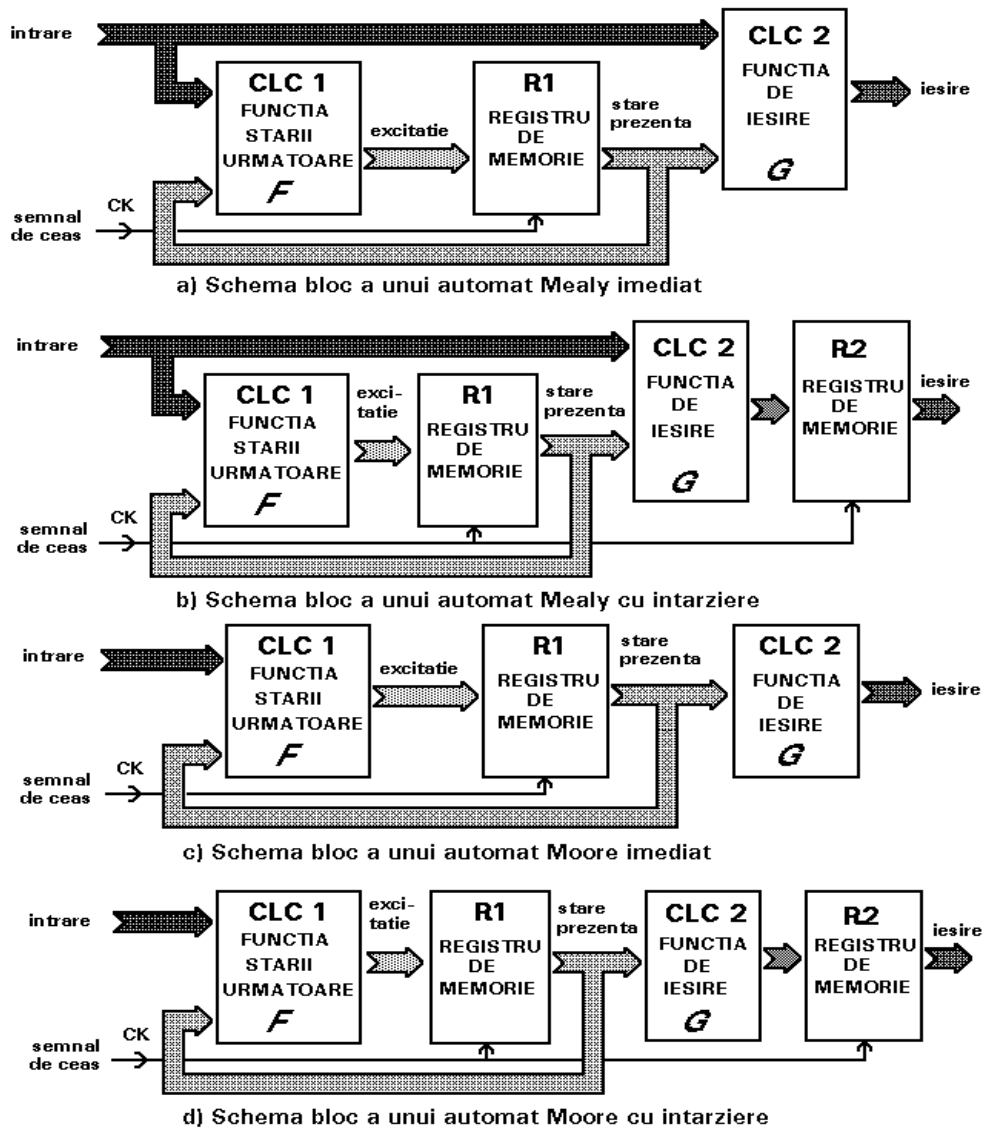


Fig. 1. Scheme bloc de principiu pentru automate de tip Mealy, respectiv Moore

2. 2. Modalități de reprezentare a automatelor

A.) Graful de tranziție a stărilor

Este o metodă intuitivă ce permite descrierea și verificarea rapidă a funcționării unui automat. Pentru fiecare nod al grafului, reprezentat printr-un cerculeț, se asociază o stare a automatului, iar fiecare arc orientat între două noduri corespunde tranziției între două stări.

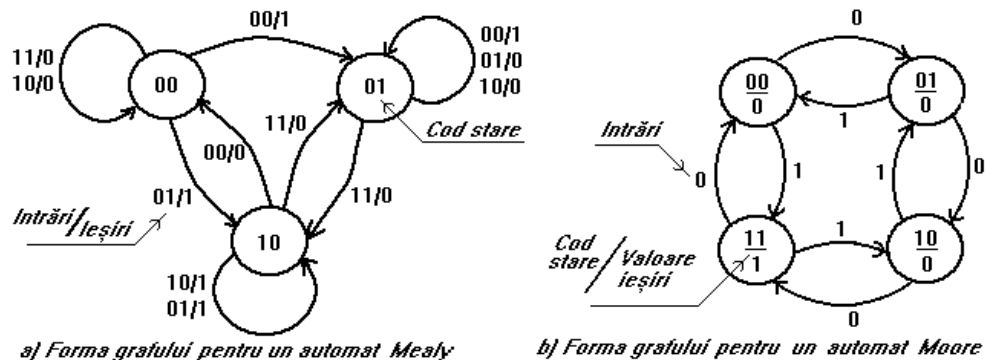


Fig. 2. Exemple de garfuri de tranziție pentru cele două tipuri de automate

Observații:

- În cazul automatelor de tip Mealy, în cerculeț se trece codul stării, iar pe arc se trec variabilele de intrare ce provoacă tranziția respectivă precum și ieșirea determinată de acestea.
- La automatele Moore, ieșirea nu depinde de stare motiv pentru care în cerculeț se trece codul stării și valoarea ieșirii, iar pe arc se trece combinația intrării ce generează tranziția.
- Pentru exemplificare, în fig. 2 se prezintă câte un graf pentru fiecare caz.

B.) Tabelul de tranziție a stărilor

Informația conținută în diagrama de tranziție poate fi transpusă într-o formă mai utilă pentru sinteza automatului, această formă poartă denumirea de tabel de tranziție. Transpunerea celor două grafuri de tranziție a stărilor din figura 2, în tabele de tranziție se arată în tabelele ce urmează.

Tabelul de tranziție a stărilor corespunzător grafului din fig. 2. b)

Stare prezentă $Q_1 Q_0$	Intrări $X_1 X_0$	Stare următoare $Q_1 Q_0$	Ieșiri Y
0 0	0 0	0 1	1
0 0	0 1	1 0	1
0 0	1 0	0 0	0
0 0	1 1	0 0	0
0 1	0 0	0 1	1
0 1	0 1	0 1	0
0 1	1 0	0 1	0
0 1	1 1	1 0	0
1 0	0 0	0 0	0
1 0	0 1	1 0	1
1 0	1 0	1 0	1
1 0	1 1	0 1	0
1 1	x x	x x	x

Tabelul de tranziție a stărilor corespunzător grafului din fig. 2. b)

Stare prezentă $Q_1 Q_0$	Intrare X	Stare viitoare $Q_1 Q_0$	Ieșire Y
0 0	0	0 1	0
0 0	1	1 1	0
0 1	0	1 0	0
0 1	1	0 0	0
1 0	0	1 1	0
1 0	1	0 1	0
1 1	0	0 0	1
1 1	1	1 0	1

2.3. Analiza automatelor elementare

Studiul funcționării unui automat elementar dat se numește **analiză**. Analiza unui automat are drept scop determinarea funcțiilor F și G astfel încât funcționarea automatului să poată fi prezisă pe baza schemei electrice a acestuia. Etapele parcurse în analiza unui automat sunt reliefate în următoarele două exemple:

Exemplul: Să se analizeze funcționarea automatului sincron din fig. 3:

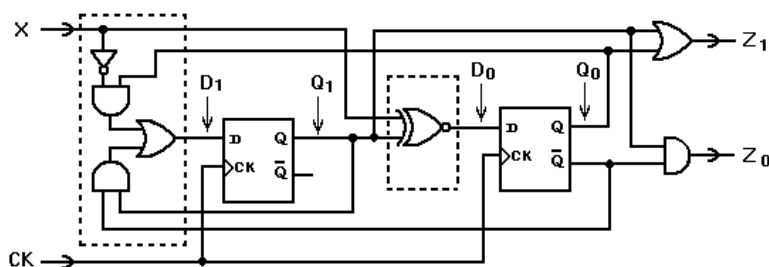


Fig. 3. Exemplu de automat Moore

Din analiza schemei electrice se observă că este vorba despre un automat Moore imediat caracterizat de: o intrare notată X ; doua ieșiri notate Z_1 , Z_0 și un registru de stare format din doi bistabili D , ceea ce înseamnă că automatul are cel mult 4 stări distincte.

Etapă	Observații
Pasul 1: Determinarea semnalelor de excitație a bistabililor Prin semnale de excitație sunt referite semnalele aplicate intrărilor pasive ale bistabililor (D pentru bistabili D, JK pentru bistabili JK, etc.). Un semnal de excitație este sintetizat cu ajutorul unui CLC ce are drept intrări starea curentă și intrările automatului. Odată delimitat acest CLC se trece la descrierea sa matematică.	Pentru automatul din fig. 3, semnale de excitație sunt calculate de cele două circuite combinatoriale delimitate cu linie punctată și au următoarele expresii logice: $D_1 = \left[\begin{array}{c c} \bar{X} & Q_0 \\ \hline Q_1 & Q_0 \end{array} \right], \quad D_0 = \left[\begin{array}{c c} \bar{X} & Q_1 \\ \hline X & Q_1 \end{array} \right]$
Pasul 2: Scrierea ecuației funcționale a bistabilului utilizat în registrul de stare al automatului.	Ecuția funcțională a unui bistabil D este dată de relația: $Q^+ = D$

	unde prin Q^+ s-a notat starea viitoare a bistabilului. În cazul bistabililor JK, ecuația funcțională este : $Q^+ = \left \frac{J \overline{Q}}{\overline{K} Q} \right $																																																																						
Pasul 3: Determinarea ecuațiilor de tranziție a stărilor Ecuațiile de tranziție a stărilor se obțin prin înlocuirea semnalelor de excitație, determinate la pasul 1, în ecuația funcțională a bistabilului scrisă în pasul 2.	Pentru exemplul considerat obținem: $Q_1^+ = D_1 = \left \frac{\overline{x} Q_0}{Q_1 Q_0} \right \qquad Q_0^+ = D_0 = \left \frac{\overline{x} Q_1}{x Q_1} \right $																																																																						
Pasul 4: Alcătuirea tabelului de tranziție Pentru fiecare stare prezentă a automatului, se determină starea următoare în care v-a ajunge automatul luând în calcul totalitatea combinațiilor variabilelor de intrare. Pentru automatul luat ca exemplu, există o singură variabilă de intrare ceea ce înseamnă că tabelul de tranziție va conține 4x2 linii.	<table><tr><th colspan="2">Stare prezentă</th><th>Intrare</th><th colspan="2">Stare viitoare</th><th colspan="2">Ie° iri</th></tr><tr><th>Q_1</th><th>Q_0</th><th>X</th><th>Q_1^+</th><th>Q_0^+</th><th>Z_1</th><th>Z_0</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	Stare prezentă		Intrare	Stare viitoare		Ie° iri		Q_1	Q_0	X	Q_1^+	Q_0^+	Z_1	Z_0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	0	1	1	0	1	0	1	1	0	0	0	1	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	0	1	0	0	1	1	1	1	0	1	0	1
Stare prezentă		Intrare	Stare viitoare		Ie° iri																																																																		
Q_1	Q_0	X	Q_1^+	Q_0^+	Z_1	Z_0																																																																	
0	0	0	0	1	0	0																																																																	
0	0	1	0	0	0	0																																																																	
0	1	0	1	1	0	1																																																																	
0	1	1	0	0	0	1																																																																	
1	0	0	1	0	1	1																																																																	
1	0	1	1	1	1	1																																																																	
1	1	0	1	0	0	1																																																																	
1	1	1	0	1	0	1																																																																	
Pasul 5: Trasarea grafului de tranziție Graful de tranziție extras din tabelul anterior ne arată că automatul din fig. 3 este un numărător binar în cod Gray care, numără înainte pentru $x=0$ și invers pentru $x=1$. În plus, odată ajuns la cap de scală procesul de numărare se oprește. Ieșirea z_1 semnalizează prin "unu logic" atingerea stării minime, iar z_0 atingerea stării maxime a numărătorului în cod Gray pe 2 biți.																																																																							

2. 4. Sinteza automatelor elementare sincrone

Proiectarea unui automat elementar, pornind de la o descriere a funcționării acestuia în cuvinte (cazul cel mai uzual și totodată cel mai dificil), sau de la o diagramă de tranziție se numește sinteză. Procesul de sinteză necesită etapelor prezentate mai jos.

• Descrierea formală a funcționării automatului

Această etapă este cea mai importantă deoarece trebuie să facă trecerea de la descrierea vagă, uneori ambiguă, a limbajului natural spre o specificație clară a funcționării automatului.

Observații:

- de regulă, sunt cunoscute numărul variabilelor de intrare și al celor de ieșire;
- stărilor automatului le sunt asociate mnemonici cât mai sugestive;
- pe parcursul descrierii pot apare probleme ce nu au fost cuprinse în formularea inițială;
- este posibil ca descrierea corectă să nu rezulte din prima iterație și atunci procesul trebuie reluat;
- rezultatul final al acestei etape este o diagramă de stare sau un tabel de stare ieșire.

• Reducerea numărului de stări ale automatului

Din etapa anterioară este posibil ca descrierea formală să conțină un număr mai mare de stări decât cel strict necesar. Reducerea numărului de stări se bazează pe identificarea stărilor echivalente. Două stări ale automatului sunt echivalente, și deci pot fi înlocuite cu una singură, dacă ele nu se pot deosebi prin inspectarea stării curente și a celei următoare. Trebuie remarcat că în procesul de comparare nu participă variabilele de stare.

Se poate demonstra că stările A și B, sunt echivalente dacă sunt îndeplinite următoarele condiții:

- A și B trebuie să conducă la aceleași valori ale variabilelor de ieșire ale automatului, pentru automatele Mealy această condiție trebuie îndeplinită pentru toate variabilele de intrare;
- pentru fiecare combinație de intrare în parte, din A și B trebuie să se ajungă în aceeași stare următoare.

Observații:

- Foarte adesea, în practică, sunt aplicații în care anumite combinații ale variabilelor de intrare nu apar niciodată, caz în care atât ieșirile cât și stările următoare ale automatului nu au nici o semnificație pentru acesta. În tabelul tranzițiilor aceste cazuri se notează cu *don't care* și sunt tratate ca în cazul simplificărilor de funcții binare.
- Două stări binare, A și B, pot fi echivalente chiar dacă nu conduc spre aceleași stări următoare numai cu condiția realizării de echivalențe între stările următoare, "ținite", de stările inițiale A și B.

- Există metode automate de reducere a stărilor însă sunt mai puțin utilizate de proiectanți, deoarece în baza experienței acumulate, aceștia pot face o descriere minimală a funcționării unui automat.
- Nu întotdeauna, un număr redus de stări înseamnă și o implementare mai simplă, există și situații în care creșterea numărului de stări poate simplifica proiectul.

• **Asignarea stărilor automatului**

În această etapă, pentru fiecare stare a automatului, stare până acum referită printr-o mnemonică, i se asociază un **cod binar unic**. Asocierea de coduri binare, pentru fiecare stare a automatului, se poate face în mod arbitrar sau urmărind un anumit criteriu. Deoarece există mai multe posibilități de codare a stărilor, vor exista tot atâtea variante de scheme logice de realizare, echivalente funcțional dar deosebite ca mod de interconectare. Este greu de estimat care schemă logică, dintre cele posibile, este cea mai redusă în complexitate.

Uzual, codarea stărilor se face cel mai adesea arbitrar, sau folosind un criteriu de optimizare cum ar fi: codarea cu variație minimă sau codarea cu dependență redusă față de intrări.

Codificarea cu variație minimă constă în asignarea unor coduri astfel încât trecerea între două stări succesive ale automatului să se facă prin modificarea unui număr cât mai redus de biți ai cuvântului de stare. Această codificare prezintă avantajul că generează expresii simple pentru funcțiile ce calculează stările următoare.

Codificarea cu dependență redusă față de intrări. Funcțiile de tranziție și cele de ieșire depind atât de starea curentă cât și de intrări. Pe căile de tranziție necondiționate nu se testează nici o variabilă de intrare, în schimb pe cele condiționate se poate ajunge la testarea tuturor variabilelor de intrare. Codificare care conduce la expresii ale funcțiilor de tranziție cu o dependență cât mai redusă față de intrări este denumită **codificare cu dependență redusă**.

Observații:

- numărul maxim al stărilor distincte ale unui automat cu n bistabili este egal cu: 2^n .
- pentru stări apropiate din punct de vedere funcțional se recomandă alocarea unor cuvinte de cod care să difere printr-un singur bit (codare cu variație minimă);
- dacă există stări nefolosite se alege cea mai convenabilă codare pentru stările utilizate, nu trebuie să ne limităm strict la codarea binară naturală;
- schimbarea codului unei stări provoacă modificarea întregii scheme logice a automatului;
- acolo unde este posibil, se recomandă separarea din setul variabilelor de stare a biților, sau a grupurilor de biți, cu semnificație precisă referitoare la schimbarea ieșirilor automatului;

• **Alcătuirea tabelului de tranziție / ieșire**

După ce s-au efectuat etapele anterioare, se optează pentru un tip de bistabil, după care se trece la alcătuirea tabelului de tranziție/ieșire. În acest tabel se trec și semnalele de excitație ai bistabililor automatului. După rezolvarea acestei etape, problema se reduce la implementarea unor funcții binare, funcțiile de excitație a bistabililor și funcțiile de ieșire ale automatului.

• **Deducerea ecuațiilor funcțiilor de excitație și a celor de ieșire**

Din tabelul rezultat la pasul anterior se extrage forma analitică a funcțiilor de excitație și a celor de ieșire, după care, prin diverse procedee se minimizează aceste expresii în vederea implementării automatului cu un hardware minimal.

• **Implementarea automatului**

Rezultatul final al acestei etape este o schemă logică care trebuie să realizeze o funcționare conformă cu diagrama automatului. Spunem o schemă logică deoarece pot fi o multitudine de realizări logice ale aceluiași automat.

2. 4. Utilizarea limbajului VHDL pentru implementarea automatelor sincrone

În mod evident, cea mai avantajoasă metodă de descriere a automatelor sincrone este cea comportamentală.

Am arătat anterior că în structura internă a unui automat putem distinge un registru de stare și două circuite logice combinatoriale: unul pentru calculul stării viitoare și altul pentru calculul ieșirii. Referitor la descrierea acestor blocuri funcționale cu ajutorul VHDL sunt valabile următoarele reguli:

- Circuitul logic de calcul al ieșirii poate fi descris fie printr-un proces special destinat acestui scop, fie prin declarații concurente.
- Registrul de stare trebuie să fie descris în mod obligatoriu printr-un proces declanșat de semnalul de ceas și eventual (dacă există) de semnalul de reset.
- Logica de calcul a stării viitoare poate fi descrisă printr-un proces separat de celelalte sau poate fi inclusă în procesul destinat registrului de stare.

Se observă așadar că există posibilitatea de a descrie automatul cu două sau cu trei procese. Din punct de vedere al ușurinței de proiectare, mai ales pentru începători, se recomandă descrierea cu trei procese. Din punct de vedere al performanțelor circuitului rezultat în urma sintezei, în literatura de specialitate se arată că este mai avantajoasă descrierea cu două procese.

Referitor la codificarea stărilor automatului, există două posibilități:

- a) **Codificarea este făcută de compilator** după niște reguli interne (de regulă, metoda implicită de codificare este cea binară iar dacă se dorește optimizarea din punct de vedere al vitezei de răspuns se folosește codificarea de tip "one hot").

Pentru declararea stărilor automatului trebuie introduse declarații similare celor de mai jos:

```

type st_val is (st0, st1, st2);
signal stare_prez, stare_viit: st_val;

```

- b) **Codificarea stărilor poate fi forțată** după dorința noastră prin intermediul unor declarații similare celor de mai jos:

```

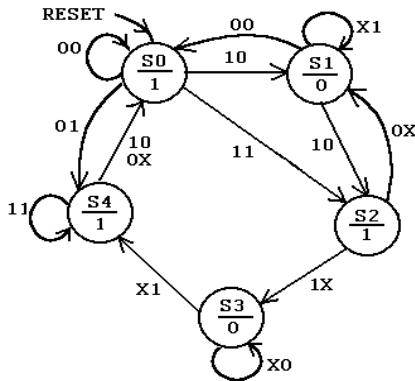
subtype stare_Automat is std_logic_vector(1 downto 0);
signal stare_prezentă, stare_viitoare :stare_Automat;
constant st0: stare_Automat := '01';
constant st1: stare_Automat := '11';
constant st2: stare_Automat := '11';

```

În cele ce urmează prezentăm câteva exemple de automate de tip Mealy și Moore descrise prin două sau trei procese cu sau fără impunerea codificării stărilor automatului.

♦ **Exemplul 1:** Descrierea în limbaj VHDL a unui automat de tip Moore (Varianta 1 - descriere folosind trei procese)

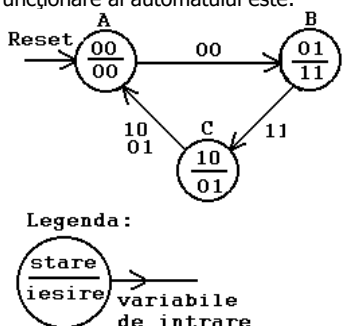
Observații	Codul VHDL
Secțiune dedicată includerii de librării	-- Exemplu de automat Moore cu 5 stări library ieee; use ieee.std_logic_1164.all;
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: moore ; - intrarea de ceas : clk ; - intrarea de ștergere: reset ; - intrările automatului: data_in ; - ieșirea automatului: data_out ; - afișare stare automat: afis_stare ;	entity moore is port (clk, reset: in std_logic; data_out: out std_logic; afis_stare: out std_logic_vector (2 downto 0); data_in: in std_logic_vector (1 downto 0)); end moore;
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: arh_comport ; - arhitectura este asociată cu entitatea: mealy ; - stările automatului sunt introduse printr-o enumerare, codificarea prpriu-zisă este lăsată pe seama compilatorului; - sunt folosite 3 procese: reg_stare pentru descrierea registrului de stare, st_viit pentru determinare stare viitoare și iesire pentru determinarea ieșirilor automatului; - graful de funcționare al automatului este:	architecture arh_comport of moore is type st_val is (st0, st1, st2, st3, st4); signal stare_prez, stare_viit: st_val; begin -- proces ptr. registrul de stare reg_stare: process (clk, reset) begin if (reset = '1') then stare_prez <= st0; elsif (clk = '1' and clk'event) then stare_prez <= stare_viit; end if; end process reg_stare; -- proces ptr. calcul stare viitoare st_viit: process (stare_prez, data_in) begin case stare_prez is when st0 => case data_in is when "00" => stare_viit <= st0; when "01" => stare_viit <= st4; when "10" => stare_viit <= st1; when "11" => stare_viit <= st2; when others => null; end case; when st1 => case data_in is when "00" => stare_viit <= st0; when "10" => stare_viit <= st2; when others => stare_viit <= st1; end case; when st2 => case data_in is when "00" => stare_viit <= st1; when "01" => stare_viit <= st1; when "10" => stare_viit <= st3; when "11" => stare_viit <= st3; when others => null; end case; when st3 => case data_in is when "01" => stare_viit <= st4; when "11" => stare_viit <= st4; when others => stare_viit <= st3; end case; end process st_viit; end architecture arh_comport;
Observații referitoare la procesul reg_stare : - este folosit pentru a descrie funcționarea registrului de stare al automatului; - lista de senzitivități cuprinde intrarea de ceas și intrarea de reset; - intrarea de reset este activă pe unu logic; - intrarea de ceas este activă pe tranziția pozitivă;	
Observații referitoare la procesul st_viit : - este folosit pentru a determina starea viitoare a automatului; - lista de senzitivități cuprinde starea prezentă și intrările automatului; - starea viitoare a automatului depinde de starea prezentă și de intrările automatului;	



- selecția după starea prezentă se face prin declarații when, iar selecția în funcție de intrările automatului se face cu declarații de tip case; Observații referitoare la procesul iesire : - este folosit pentru determinarea ieșirii automatului pe baza stării prezente a automatului; - lista de senzitivități cuprinde doar starea prezentă a automatului; Observații referitoare la afișarea stării automatului: - afișarea stării este necesară doar din considerente didactice; - în acest exemplu, codificarea stărilor este făcută de compilator – în consecință nu știm ce asocieri a făcut acesta; - propunem următoarea combinație pentru a vizualiza schimbarea stărilor (este ca și cum am calcula un nou set de ieșiri din automat): - st0 → 111; - st1 → 001; - st2 → 010; - st3 → 011; - st4 → 100;	<pre> end case; when st4 => case data_in is when "11" => stare_viit <= st4; when others => stare_viit <= st0; end case; when others => stare_viit <= st0; end case; end process st_viit; -- proces ptr. calcul iesiri -- depind doar de starea prezenta iesire: process (stare_prez) begin case stare_prez is when st0 => data_out <= '1'; afis_stare <="111"; when st1 => data_out <= '0'; afis_stare <="001"; when st2 => data_out <= '1'; afis_stare <="010"; when st3 => data_out <= '0'; afis_stare <="011"; when st4 => data_out <= '1'; afis_stare <="100"; when others => data_out <= '0'; afis_stare <="110"; end case; end process iesire; end arh_comport; </pre>
Fișierul de constrângeri	<pre> NET "clk" LOC = "P56"; NET "reset" LOC = "P83"; NET "data_in<1>" LOC = "P37"; NET "data_in<0>" LOC = "P40"; NET "afis_stare<2>" LOC = "P62"; NET "afis_stare <1>" LOC = "P65"; NET "afis_stare <0>" LOC = "P67"; NET "data_out" LOC = "P82"; </pre>

◆ **Exemplul 2:** Descrierea în limbaj VHDL a unui automat de tip Moore (Varianta 2 - descriere folosind două procese)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre> -- Exemplu de automat Moore cu 5 stari library ieee; use ieee.std_logic_1164.all; </pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: moore; - intrarea de ceas : clk; - intrarea de ștergere: reset; - intrările automatului: data_in; - ieșirea automatului: data_out; - afișare stare automat: afis_stare;	<pre> entity moore is port (clk, reset: in std_logic; data_out: out std_logic(1 downto 0); afis_stare: out std_logic(1 downto 0); data_in: in std_logic_vector (1 downto 0)); end moore; </pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: - numele arhitecturii este: arh_comport ; - arhitectura este asociată cu entitatea: moore; - sunt folosite 2 procese; Observații referitoare la procesul reg_stare : - este folosit pentru a descrie funcționarea registrului de stare al automatului; - lista de senzitivități cuprinde intrarea de ceas și intrarea de reset; - intrarea de reset este activă pe zero logic; - intrarea de ceas este activă pe tranziția pozitivă;	<pre> architecture arh_comport of moore is subtype st_val is std_logic_vector(1 downto 0); signal stare_prez, stare_viit: st_val; constant stA : st_val := '00'; constant stB : st_val := '01'; constant stC : st_val := '10'; begin -- proces ptr. registrul de stare reg_stare: process (clk, reset) begin if (reset = '1') then stare_prez <= stA; elsif (clk = '1' and clk'event) then stare_prez <= stare_viit; end if; end process reg_stare; -- proces ptr. CLC comb: process (stare_prez, data_in) begin stare_viit <= stare_prez; </pre>

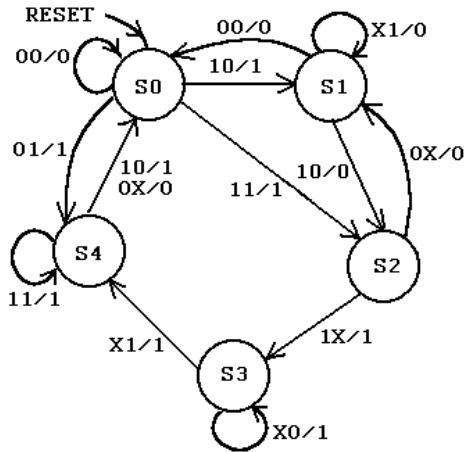
Graful de funcționare al automatului este:  Legenda: Observații referitoare la procesul comb: - este folosit pentru a determina starea viitoare a automatului; - lista de sensibilități cuprinde starea prezentă și intrările automatului; - starea viitoare a automatului depinde de starea prezentă și de intrările automatului; Observații referitoare la calculul ieșirilor: - se folosesc două asignări concurente, acestea se desfășoară în paralel cu procesele. Observații referitoare la afișarea stării automatului: - afișarea stării este necesară doar din considerente didactice; - în acest exemplu, codificarea stărilor este impusă de noi, deci este cunoscută;	<pre> case stare_prez is when stA => if data_in="00" then stare_viit <= stB; end if; when stB => if data_in="11" then stare_viit <= stC; end if; when stC => if data_in="01" or data_in="10" then stare_viit <= stA; end if; when others => stare_viit <= stA; end case; end process comb; -- asignari cocurente ptr. iesiri data_out(1)<='1' when stare_prez =stB else '0'; data_out(0)<='0' when stare_prez =stA else '1'; afis_stare<= stare_prez; -- afisare stare end arh_comport; </pre>
Fișierul de constrângeri:	<pre> NET "clk" LOC = "P56"; NET "reset" LOC = "P83"; NET "data_in<1>" LOC = "P37"; NET "data_in<0>" LOC = "P40"; NET "afis_stare<1>" LOC = "P65"; NET "afis_stare<0>" LOC = "P67"; NET "data_out<1>" LOC = "P80"; NET "data_out<0>" LOC = "P82"; </pre>

◆ **Exemplul 3:** Descrierea în limbaj VHDL a unui automat de tip Mealy (descriere folosind trei procese)

Observații	Codul VHDL
Secțiune dedicată includerii de librării	<pre> -- Example of a 5-state Mealy FSM library ieee; use ieee.std_logic_1164.all; </pre>
Secțiune dedicată descrierii entității. În cazul de față: - nume entitate este: mealy; - intrarea de ceas: clk; - intrarea de ștergere: reset; - intrările automatului: data_in; - ieșirea automatului: data_out;	<pre> entity mealy is port (clk, reset: in std_logic; data_out: out std_logic; afis_stare: out std_logic_vector(1 downto 0); data_in: in std_logic_vector (1 downto 0)); end mealy; </pre>
Secțiune dedicată descrierii arhitecturii. În cazul de față: - sunt folosite 3 procese: reg_stare pentru descrierea registrului de stare, st_viit pentru determinarea stării viitoare și iesire pentru determinarea ieșirilor automatului; Observații referitoare la procesul reg_stare: - este folosit pentru a descrie funcționarea registrului de stare al automatului; - lista de sensibilități cuprinde intrarea de ceas și	<pre> architecture arh_comport of mealy is type st_val is (st0, st1, st2, st3, st4); signal stare_prez, stare_viit: st_val; begin -- proces ptr. registrul de stare reg_stare: process (clk, reset) begin if (reset = '1') then stare_prez <= st0; elsif (clk'event and clk = '1') then stare_prez <= stare_viit; </pre>

- intrarea de reset;
- intrarea de reset este activă pe zero logic;
- intrarea de ceas este activă pe tranziția pozitivă;

Graful de funcționare al automatului este:



Observații referitoare la procesul **st_viit** :

- este folosit pentru a determina starea viitoare a automatului;
- lista de sensibilități cuprinde starea prezentă și intrările automatului;
- starea viitoare a automatului depinde de starea prezentă și de intrările automatului;
- selecția după starea prezentă se face prin declarații **when**, iar selecția în funcție de intrările automatului se face cu declarații de tip **case**;

Observații referitoare la procesul **iesire** :

- este folosit pentru determinarea ieșirii automatului pe baza stării prezente și a intrărilor automatului;
- lista de sensibilități cuprinde starea prezentă și intrările automatului;

Observații referitoare la afișarea stării automatului:

- afișarea stării este necesară doar din considerente didactice;
- codificarea binară a stărilor **S0, S1, S2, S3, S4** este lăsată pe seama compilatorului de VHDL;
- propunem următoarea combinație pentru a vizualiza schimbarea stărilor (este ca și cum am calcula un nou set de ieșiri din automat):
 - S0 → 111; S1 → 001; S2 → 010;
 - S3 → 011; S4 → 100;

```

end if;
end process reg_stare;

-- proces ptr. determinare stare viitoare
st_viit: process (stare_prez, data_in)
begin
  case stare_prez is
    when st0 =>
      case data_in is
        when "00" => stare_viit<= st0;
        when "01" => stare_viit<= st4;
        when "10" => stare_viit<= st1;
        when "11" => stare_viit<= st2;
        when others => null;
      end case;
    when st1 =>
      case data_in is
        when "00" => stare_viit<= st0;
        when "10" => stare_viit<= st2;
        when others => stare_viit<= st1;
      end case;
    when st2 =>
      case data_in is
        when "00" => stare_viit<= st1;
        when "01" => stare_viit<= st1;
        when "10" => stare_viit<= st3;
        when "11" => stare_viit<= st3;
        when others => null;
      end case;
    when st3 =>
      case data_in is
        when "01" => stare_viit<= st4;
        when "11" => stare_viit<= st4;
        when others => stare_viit<= st3;
      end case;
    when st4 =>
      case data_in is
        when "11" => stare_viit<= st4;
        when others => stare_viit<= st0;
      end case;
    when others => stare_viit<= st0;
  end case;
end process st_viit;

-- proces ptr. calcul iesire din automat
iesire: process (stare_prez, data_in)
begin
  case stare_prez is
    when st0 =>
      afis_stare<= "000";
      case data_in is
        when "00" => data_out <= '0';
        when others => data_out <= '1';
      end case;
    when st1 => data_out <= '0';
      afis_stare<= "001";
    when st2 =>
      afis_stare<= "010";
      case data_in is
        when "00" => data_out <= '0';
        when "01" => data_out <= '0';
        when others => data_out <= '1';
      end case;
    when st3 => data_out <= '1';
      afis_stare<= "011";
    when st4 =>
      afis_stare<= "100";
      case data_in is
        when "10" => data_out <= '1';
        when "11" => data_out <= '1';
        when others => data_out <= '0';
      end case;
  end case;
end process iesire;
  
```

	<pre> when others => data_out <= '0'; afis_stare<= "110"; end case; end process iesire; end arh_comport; </pre>
Fișierul de constrângeri:	<pre> NET "clk" LOC = "P56"; NET "reset" LOC = "P83"; NET "data_in<1>" LOC = "P37"; NET "data_in<0>" LOC = "P40"; NET "afis_stare<2>" LOC = "P62"; NET "afis_stare<1>" LOC = "P65"; NET "afis_stare<0>" LOC = "P67"; NET "data_out" LOC = "P82"; </pre>



3. Desfășurarea lucrării

3.1. Analiza automatelor sincrone.

- A. Folosind metode similare celor prezentate în partea teoretică, determinați grafurile de tranziție pentru automatele din figura 6.

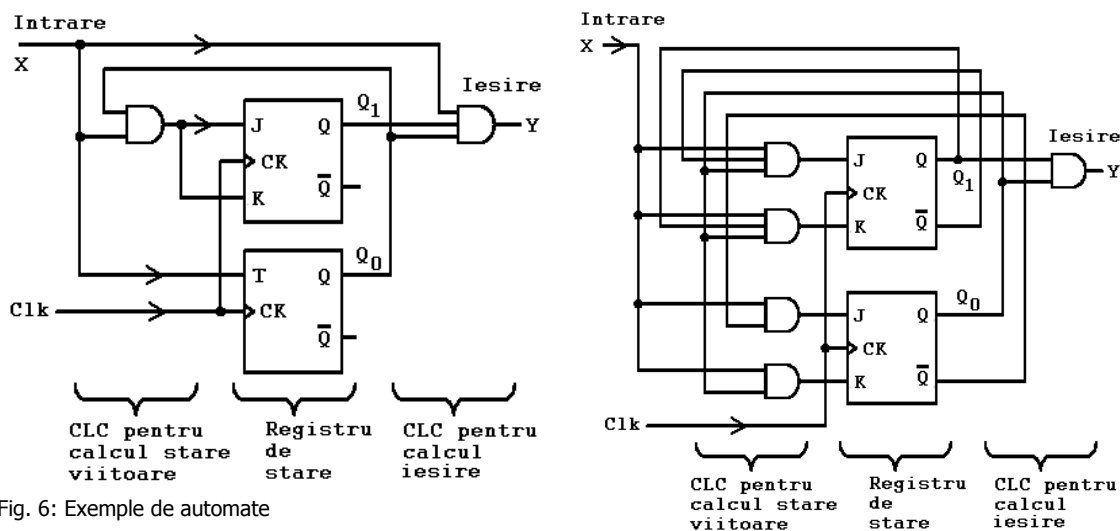


Fig. 6: Exemple de automate

3.2. Sinteza automatelor sincrone.

- A. Folosind metodologia de proiectare a automatelor sincrone determinați schema logică a unui numărător sincron bidirecțional pe 3 biți.
 B. Pentru automatele descrise în exemplele VHDL, determinați câte o schemă logică folosind bistabili la alegere.

3.3. Aplicații cu VHDL.

- A. Verificați pe macheta de laborator cu CPLD, descrierile VHDL prezentate ca exemple.
 B. Descrieți în limbaj VHDL automatele sincrone ce funcționează după grafurile de tranziție din figura 2.
 C. Folosind facilitatea ISE WebPack prin care se permite introducerea schemelor logice ale sistemelor digitale, implementați schemele din figura 6 și comparați rezultatele obținute cu cele de la subpunctul 3.1. din desfășurarea lucrării.

Mode de lucru:

- variabilele de intrare vor fi implementate cu switch-uri de pe macheta de laborator;
- starea automatului se afișează în cod binar pe două sau trei LED-uri existente pe macheta de laborator;
- starea semnalelor de ieșire se afișează în cod binar pe alte LED-uri (rămase libere) de pe macheta de laborator;
- semnalul de ceas al automatului se preia de la oscilatorul de 25,175MHz de pe macheta de laborator, eventual printr-o divizare prealabilă a frecvenței.

